

NOTICE Deployment Manual

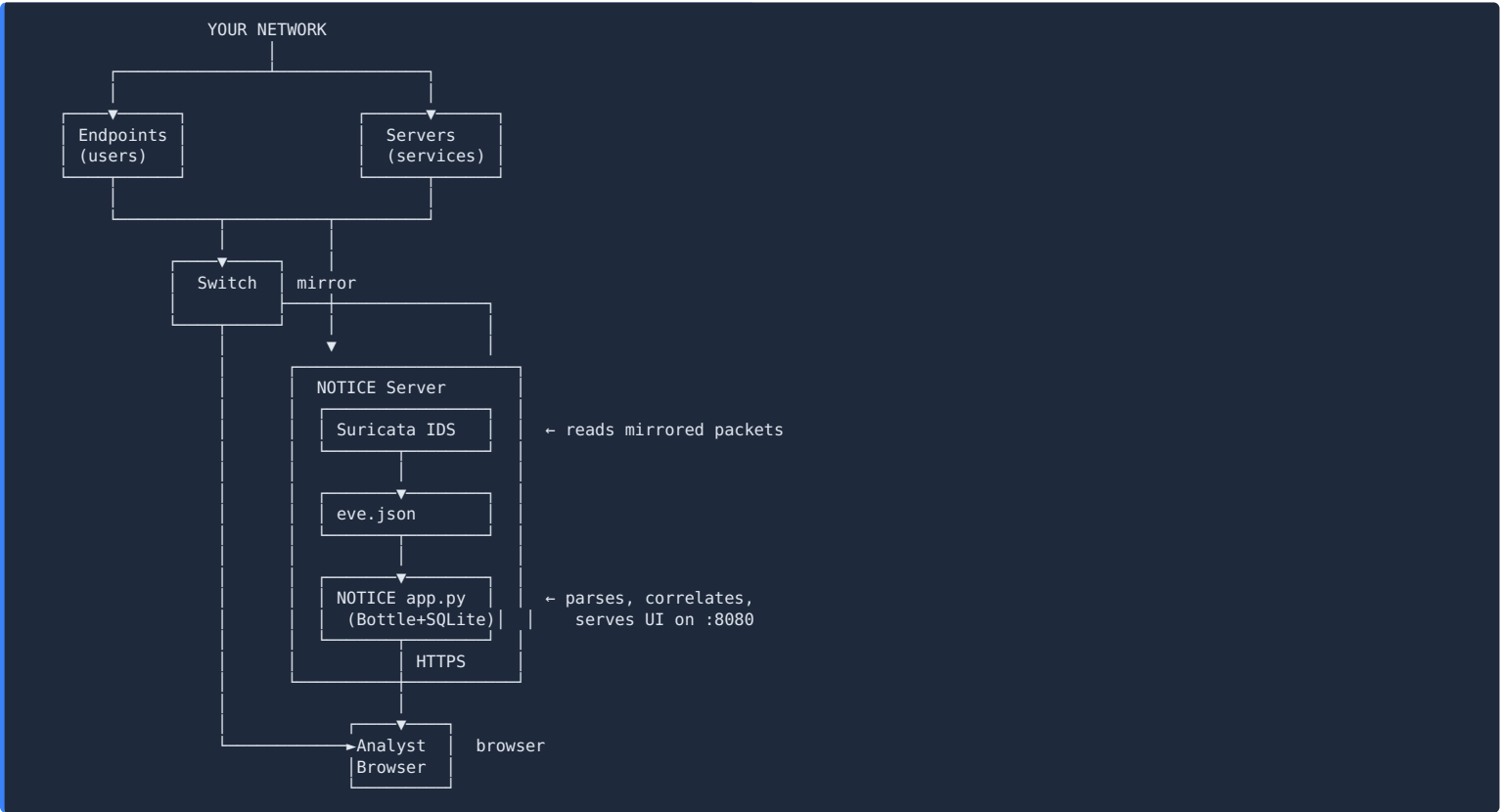
Complete step-by-step guide to deploy the NOTICE security monitoring platform from a blank Ubuntu server to a fully operational SOC console.

Target audience: Sysadmins / SOC engineers deploying NOTICE in an organization. **Target OS:** Ubuntu 22.04 LTS or 24.04 LTS (Debian 12 works identically). Other distros need dependency adjustments. **Estimated time:** 60–90 minutes for a first full install.

Table of Contents

- 1. Architecture Overview
- 2. Prerequisites & Sizing
- 3. Network Setup — SPAN / Mirror Port
- 4. OS Preparation
- 5. Suricata Installation & Configuration
- 6. Integrating with an Existing Suricata Instance
- 7. NOTICE Application Installation
- 8. First-Time Configuration
- 9. Running NOTICE as a Systemd Service
- 10. Post-Install Tasks
- 11. Operational Procedures
- 12. Troubleshooting
- 13. Security Hardening
- 14. Optional: External Enrichment (VT / AbuseIPDB / GeoIP)

1. Architecture Overview



Two network interfaces on the NOTICE server:

- **Management interface** (e.g., `eth0`) — used for SSH, NOTICE web UI, package updates. Has a real IP.
- **Monitoring interface** (e.g., `eth1`) — receives mirrored network traffic. Usually has NO IP address (put in promiscuous mode).

2. Prerequisites & Sizing

Hardware minimums

Workload	CPU	RAM	Disk	Network
Small lab (<100 hosts, <10 Mbps)	2 cores	4 GB	50 GB SSD	2 × 1 Gbps NIC
Small org (100–500 hosts, <100 Mbps)	4 cores	8 GB	200 GB SSD	2 × 1 Gbps NIC
Medium org (500–2000 hosts, <500 Mbps)	8 cores	16 GB	500 GB SSD	2 × 10 Gbps NIC
Large (>2000 hosts, >1 Gbps)	16+ cores	32+ GB	1 TB+ NVMe	2 × 10 Gbps NIC + tuning

- **Disk grows fast:** Suricata’s `eve.json` can hit 1–10 GB per day on a small org. Plan retention.
- **SSD strongly recommended** — SQLite WAL mode and eve.json tailing are I/O-heavy.

Software prerequisites

- Ubuntu 22.04 / 24.04 LTS (or Debian 12+)
- `sudo` access
- Internet access for package installs

3. Network Setup — SPAN / Mirror Port

Suricata is a passive IDS. It cannot see traffic unless the switch feeds it a copy. There are three common options:

Option A — Switch SPAN / mirror port (recommended for organizations)

Configure your managed switch to mirror the traffic you want to monitor to the port connected to the NOTICE server’s monitoring NIC.

Cisco example (monitors VLAN 10 → port Gi0/24):

```
switch# configure terminal
switch(config)# monitor session 1 source vlan 10 both
switch(config)# monitor session 1 destination interface Gi0/24
switch(config)# end
switch# copy running-config startup-config
```

HP/Aruba example:

```
config
mirror 1 port 24
interface 1-23
    monitor all both mirror 1
exit
```

Consult your switch vendor’s docs for the exact syntax.

Option B — Network TAP (best-quality but costs money)

A hardware tap (e.g., Garland, Profitap, Dualcomm) physically splits the fibre or copper and hands two clean copies (both directions) to your monitoring NIC. Higher fidelity than SPAN, doesn’t drop packets under load.

Option C — Monitor local traffic only (small lab / home)

If you only want to see traffic to/from the NOTICE server itself (e.g., you’re using it as a honeypot or just testing), you can monitor its own primary interface. No switch config needed. Skip mirror-port setup — Suricata will listen on the management interface directly.

Verify traffic reaches the monitoring interface

Before installing Suricata, confirm packets arrive:

```
sudo apt install -y tcpdump
sudo ip link set eth1 up promisc on    # replace eth1 with your monitoring NIC
sudo tcpdump -i eth1 -c 20 -nn
```

You should see packets from various sources. If you see only broadcast (ARP, mDNS), the mirror isn’t feeding you unicast — go back to your switch config.

4. OS Preparation

Update the system and install common tools:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl wget git tcpdump net-tools jq \
    software-properties-common ca-certificates \
    build-essential python3 python3-venv python3-dev \
    sqlite3 openssl unzip
```

Optional: set the timezone to your local TZ so timestamps in the UI match reality:

```
sudo timedatectl set-timezone Asia/Kolkata    # or your zone
```

Optional: increase file descriptor limits (busy Suricata + NOTICE need many open files):

```
echo '* soft nofile 65535' | sudo tee -a /etc/security/limits.conf
echo '* hard nofile 65535' | sudo tee -a /etc/security/limits.conf
```

5. Suricata Installation & Configuration

5.1 Install Suricata

Ubuntu's default Suricata is usually outdated. Use the OISF PPA for a current stable release:

```
sudo add-apt-repository -y ppa:oisf/suricata-stable
sudo apt update
sudo apt install -y suricata suricata-update
```

Verify:

```
suricata --build-info | head -20
sudo systemctl status suricata    # will be running but not yet correctly configured
```

5.2 Identify your monitoring interface

```
ip -brief link show
```

Note the name of your monitoring NIC (something like `eth1`, `enp0s8`, `ens192`). We'll call it `<MON_IFACE>` below.

5.3 Configure Suricata

Edit `/etc/suricata/suricata.yaml`:

```
sudo cp /etc/suricata/suricata.yaml /etc/suricata/suricata.yaml.orig
sudo nano /etc/suricata/suricata.yaml
```

Change these key settings:

5.3.1 HOME_NET (your internal address space)

Find the `vars:` block. Set `HOME_NET` to a CIDR that describes what “internal” means for you. Examples:

```
vars:
  address-groups:
    HOME_NET: "[10.0.0.0/8]"                # RFC1918 class A
    # HOME_NET: "[192.168.0.0/16]"          # small org
    # HOME_NET: "[10.0.0.0/8,192.168.0.0/16,172.16.0.0/12]" # everything private
    EXTERNAL_NET: "!$HOME_NET"
```

Whatever isn't `HOME_NET` becomes `EXTERNAL_NET`. Rules are directional (`$EXTERNAL_NET → $HOME_NET`), so this matters.

5.3.2 af-packet — capture interface

Find the `af-packet:` block. Replace the default interface with yours:

```
af-packet:
- interface: <MON_IFACE>
  cluster-id: 99
  cluster-type: cluster_flow
  defrag: yes
  checksum-checks: no    # disable if NIC does checksum offload (common)
```

Important: If your NOTICE server ALSO monitors its own primary interface (small-lab scenario), add a second entry:

```
af-packet:
- interface: eth1          # mirror port
  cluster-id: 99
  cluster-type: cluster_flow
  defrag: yes
  checksum-checks: no
- interface: eth0          # this machine's own interface
  cluster-id: 98
  cluster-type: cluster_flow
  defrag: yes
  checksum-checks: no

- interface: default
  # default settings for anything else
```

5.3.3 Global checksum validation

Scroll to the `stream:` block, disable checksum validation globally (NICs offload checksums, so Suricata otherwise drops many packets):

```
stream:
  memcap: 64 MiB
  checksum-validation: no      # was 'yes' — disable
  inline: auto
```

5.3.4 Verify eve.json output is enabled

Find `outputs:` → `- eve-log:`. Make sure it's enabled with these types:

```
outputs:
- eve-log:
  enabled: yes
  filetype: regular
  filename: eve.json
  types:
  - alert
  - flow
  - dns
  - http
  - tls
  - ssh
  - dhcp
  - files
  - stats
```

5.4 Put the monitoring interface in promiscuous mode + disable offloading

Suricata needs to see all packets, not just its own MAC. Also disable NIC-level TCP segmentation offload (otherwise Suricata sees huge super-packets that its parsers can't handle):

```
sudo ip link set <MON_IFACE> up promisc on
for feature in gro gso tso lro; do
  sudo ethtool -K <MON_IFACE> $feature off
done
```

Make it persistent across reboots — create `/etc/systemd/system/suricata-iface.service`:

```
[Unit]
Description=Prepare Suricata monitoring interface
Before=suricata.service
After=network.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/sbin/ip link set <MON_IFACE> up promisc on
ExecStart=/sbin/ethtool -K <MON_IFACE> gro off
ExecStart=/sbin/ethtool -K <MON_IFACE> gso off
ExecStart=/sbin/ethtool -K <MON_IFACE> tso off
ExecStart=/sbin/ethtool -K <MON_IFACE> lro off

[Install]
WantedBy=multi-user.target
```

Enable and start:

```
sudo systemctl daemon-reload
sudo systemctl enable --now suricata-iface.service
```

5.5 Install rule sets

`suricata-update` fetches rule packs from Emerging Threats and other sources:

```
sudo suricata-update update-sources
sudo suricata-update enable-source et/open      # free ET Open ruleset
sudo suricata-update
```

To add other sources later:

```
sudo suricata-update list-sources      # see catalogue
sudo suricata-update enable-source oisf/trafficid # example
sudo suricata-update                  # re-download all enabled
```

Rules land in `/var/lib/suricata/rules/suricata.rules` (merged single file).

Add your own custom rules to `/var/lib/suricata/rules/local.rules` — this file is **not shipped with NOTICE** (it's gitignored) because it should be tuned to YOUR network.

Example custom `local.rules` starter:

```
# Custom NOTICE rules – tune to your network

# SSH brute force to any internal host (3+ SYNs from same source in 2 min)
alert tcp !$HOME_NET any -> $HOME_NET 22 (msg:"CUSTOM SSH Brute Force from External"; flow:to_server; flags:S,12; threshold:type both, track
by_src, count 3, seconds 120; classtype:attempted-admin; sid:9000001; rev:1;)

# RDP brute force
alert tcp !$HOME_NET any -> $HOME_NET 3389 (msg:"CUSTOM RDP Brute Force from External"; flow:to_server; flags:S,12; threshold:type both, track
by_src, count 3, seconds 120; classtype:attempted-admin; sid:9000002; rev:1;)
```

Reference `local.rules` in `/etc/suricata/suricata.yaml`:

```
rule-files:
- suricata.rules
- local.rules
```

5.6 Validate and start Suricata

Test the config before starting:

```
sudo suricata -T -c /etc/suricata/suricata.yaml
```

Expected: `Configuration provided was successfully loaded. Exiting.`

Any rules errors: fix them or comment out the offending rule with `#`.

Start / restart:

```
sudo systemctl restart suricata
sudo systemctl enable suricata
sudo systemctl status suricata
```

Confirm alerts start flowing after a few minutes:

```
sudo tail -f /var/log/suricata/eve.json | grep '"event_type":"alert"'
```

You should see JSON alert lines as traffic triggers rules. If you see only `flow`, `dns`, `http` events but no `alert`, either your rules aren't loaded or the monitored traffic is benign — try generating a test alert:

```
curl http://testmyids.com/      # legit test URL that fires an ET rule
```

6. Integrating with an Existing Suricata Instance

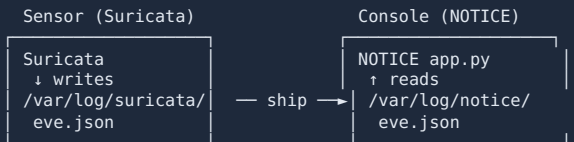
If your organization already runs Suricata (managed by another team, or on a dedicated sensor appliance), you can skip §5 and connect NOTICE to that existing instance. NOTICE only needs three things: read access to `eve.json`, correct `HOME_NET` awareness, and a compatible eve output configuration.

6.1 Deployment topologies

There are three ways to integrate depending on where Suricata runs relative to where you install NOTICE:

Topology A — Same host (Suricata + NOTICE on one server) The simplest case. NOTICE reads `/var/log/suricata/eve.json` directly. Just point NOTICE at the path and ensure file permissions allow it.

Topology B — Separate sensor and console (Suricata on sensor, NOTICE on console) Suricata runs on a dedicated sensor appliance next to your SPAN port. NOTICE runs on a separate server (usually alongside your other SOC tools). Ship `eve.json` from sensor to console over the network.



Topology C — Multiple sensors, one console Several Suricata sensors feeding a single NOTICE instance. Merge each sensor’s eve.json into one file on the NOTICE server (or point NOTICE at a fan-in log via Filebeat / rsyslog).

6.2 Sanity-check the existing Suricata

On the Suricata host, verify:

```
# 1. Suricata is running and healthy
sudo systemctl status suricata

# 2. eve.json exists and is being written
ls -lh /var/log/suricata/eve.json
sudo tail -1 /var/log/suricata/eve.json | python3 -m json.tool | head -20

# 3. Alerts are being generated (or you need to load rules)
sudo grep -c '"event_type":"alert"' /var/log/suricata/eve.json | head -1
```

If `eve.json` doesn’t exist or lacks alerts, the existing Suricata isn’t producing what NOTICE needs.

6.3 Confirm eve.json has the required event types

NOTICE parses these event types: `alert`, `flow`, `dns`, `http`, `tls`, `ssh`, `dhcp`, `files`. Check the existing Suricata’s config:

```
sudo grep -A 30 '^outputs:' /etc/suricata/suricata.yaml | grep -A 20 'eve-log:' | head -25
```

You should see something like:

```
outputs:
- eve-log:
  enabled: yes
  filename: eve.json
  types:
  - alert
  - flow
  - dns
  ...
```

If any of these types are missing, ask the Suricata admin to enable them, then reload Suricata (`sudo systemctl restart suricata`). NOTICE works even with only `alert` events, but you lose the flow / protocol enrichment.

6.4 Confirm HOME_NET matches your monitored network

NOTICE has its own idea of what counts as “internal” (default `10.0.0.0/8`, override with `MONITORED_NET` env var — see §8.1). Suricata’s `HOME_NET` should match, otherwise:

- If Suricata’s `HOME_NET` is too narrow → alerts about external attacks against your assets won’t fire.
- If NOTICE’s `MONITORED_NET` doesn’t match Suricata’s → the “protected asset” filter and internal-vs-external classification will be wrong.

Check Suricata’s `HOME_NET`:

```
sudo grep -E '^s*HOME_NET' /etc/suricata/suricata.yaml
```

Set NOTICE’s `MONITORED_NET` in `.env` to match (or a subset of it if you only care about a specific VLAN).

6.5 Topology A — same host

Just make sure the `notice` user can read `eve.json`:

```
# Add notice to the suricata group
sudo usermod -aG suricata notice

# Ensure eve.json is group-readable
sudo chmod 640 /var/log/suricata/eve.json
sudo chmod g+r /var/log/suricata          # directory too

# Verify
sudo -u notice head -1 /var/log/suricata/eve.json | head -c 100
```

Then in NOTICE’s `.env`:

```
EVE_LOG=/var/log/suricata/eve.json
```

Skip to §7 (NOTICE Application Installation).

6.6 Topology B — separate sensor and console

Option B1 — Real-time ship with Filebeat (recommended)

On the sensor, install and configure Filebeat to ship eve.json to the console:

```
# On the sensor
curl -L -O https://artifacts.elastic.co/downloads/beats/filebeat/filebeat-8.15.0-linux-x86_64.tar.gz
tar xf filebeat-8.15.0-linux-x86_64.tar.gz
cd filebeat-8.15.0-linux-x86_64
```

Edit `filebeat.yml`:

```
filebeat.inputs:
- type: filestream
  id: suricata-eve
  paths:
    - /var/log/suricata/eve.json

output.logstash:
  hosts: ["console-ip:5044"]
```

On the console, run a lightweight logstash or a simple tcp-to-file listener that appends every received line to `/var/log/notice/eve.json`.

Option B2 — rsync every N seconds (simple, slight lag)

On the console, pull the file from the sensor over SSH:

```
sudo mkdir -p /var/log/notice
sudo chown notice:notice /var/log/notice

# Cron job on the console (as notice user):
crontab -e -u notice
```

Add:

```
* * * * * rsync -az --append sensor-user@sensor-host:/var/log/suricata/eve.json /var/log/notice/eve.json
```

`--append` means only new lines are transferred each minute. Very cheap.

Set NOTICE's `.env`:

```
EVE_LOG=/var/log/notice/eve.json
```

Option B3 — Suricata direct-to-remote via redis / kafka / syslog

Suricata can emit eve events directly to redis, kafka, or syslog. On the console, run a small consumer that appends to a local file, then point NOTICE at that file. This is highest-throughput for very busy sensors — but overkill for most orgs.

Refer to Suricata's `eve-log` documentation for the redis/kafka output config.

6.7 Topology C — multiple sensors, one console

Each sensor ships to a distinct file on the console:

```
/var/log/notice/sensor-a-eve.json
/var/log/notice/sensor-b-eve.json
/var/log/notice/sensor-c-eve.json
```

Merge them into one stream that NOTICE tails:

```
sudo tee /etc/systemd/system/eve-merge.service <<'EOF'
[Unit]
Description=Merge multiple sensor eve.json streams
After=network.target

[Service]
Type=simple
User=notice
ExecStart=/bin/bash -c 'tail -F /var/log/notice/sensor-*-eve.json > /var/log/notice/eve.json'
Restart=on-failure

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now eve-merge.service
```

Point NOTICE at the merged file:

```
EVE_LOG=/var/log/notice/eve.json
```

NOTICE's pipeline handles the byte-offset bookmark on this merged file exactly like a single eve.json.

6.8 Adjusting Suricata to work better with NOTICE (optional)

If you have the freedom to tweak the existing Suricata's config, these changes improve NOTICE's value:

1. **Enable all event types** (as listed in §6.3) — richer alert context.
2. **Add community-id** to eve output — lets NOTICE correlate alerts with flows across event types:

```
outputs:
- eve-log:
  community-id: yes
  community-id-seed: 0
```

3. **Enable PCAP output** with time-based rotation — lets NOTICE offer per-alert PCAP retrieval:

```
outputs:
- pcap-log:
  enabled: yes
  filename: log.pcap
  limit: 1GB
  max-files: 100
  mode: normal
  use-stream-depth: no
  honor-pass-rules: no
```

None of these are required for NOTICE to work — but each unlocks additional analyst workflows.

6.9 Testing the integration

Once NOTICE is installed (next section), verify integration end-to-end:

```
# On the sensor, or same host: generate a benign test alert
curl http://testmyids.com/

# Within 30 seconds, on the console:
sudo tail -f /var/log/notice/eve.json | grep 'testmyids\|GPL ATTACK_RESPONSE'
```

You should see the test alert arrive. Then in the NOTICE UI (Alerts tab), it should appear within another few seconds (pipeline polls every 2 seconds).

If the alert appears in the eve.json file but not in the UI, see §12 troubleshooting.

7. NOTICE Application Installation

7.1 Create a dedicated user (optional but recommended)

```
sudo useradd -m -s /bin/bash notice
sudo usermod -aG suricata notice      # so notice user can read eve.json
```

Log in as the notice user for the rest of the install:

```
sudo -iu notice
```

7.2 Clone the repo

```
cd ~
git clone https://github.com/credninja/notice.git
cd notice
```

7.3 Python virtual environment

```
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip

# NOTICE runs on stdlib only. The only optional dep is PyYAML for Sigma rule
# import; install if you plan to use Sigma:
pip install PyYAML
```

7.4 TLS certificate (self-signed for lab; use real cert for prod)

```
mkdir -p certs
openssl req -x509 -newkey rsa:4096 -sha256 -days 3650 -nodes \
  -keyout certs/key.pem -out certs/cert.pem \
  -subj "/CN=notice.local" \
  -addext "subjectAltName=DNS:notice.local,DNS:localhost,IP:127.0.0.1"
chmod 600 certs/key.pem
```

For production, replace with a proper CA-signed cert (Let's Encrypt via a reverse proxy, or your org's PKI). See §13 for reverse-proxy setup.

7.5 Grant read access to eve.json

The notice user must be able to read Suricata's log:

```
sudo chmod 640 /var/log/suricata/eve.json
sudo chmod g+r /var/log/suricata # directory
```

If you didn't add notice to the suricata group in 7.1:

```
sudo usermod -aG suricata notice
```

Then log out and back in for group membership to apply.

7.6 Verify the install can start

```
cd ~/notice
source venv/bin/activate
python3 app.py
```

Expected output:

```
NOTICE - Network Security Monitor
Monitoring subnet: 10.0.0.0/8
Reading logs from: /var/log/suricata/eve.json
Starting server on https://0.0.0.0:8080
TLS cert: /home/notice/notice/certs/cert.pem
```

If you see errors about missing modules, ensure the venv is activated. Ctrl+C to stop for now.

8. First-Time Configuration

8.1 Environment variables

Create `.env` in the repo root:

```
cd ~/notice
cat > .env <<'EOF'
# — Core —
EVE_LOG=/var/log/suricata/eve.json
HOST=0.0.0.0
PORT=8080

# — Authentication —
AUTH_ENABLED=true

# — Optional threat intel API keys (leave blank to disable) —
# VIRUSTOTAL_API_KEY=
# ABUSEIPDB_KEY=

# — Optional custom monitored subnet (defaults to 10.0.0.0/8) —
# MONITORED_NET=192.168.0.0/16
EOF
chmod 600 .env
```

8.2 Start the server

```
source venv/bin/activate
python3 app.py
```

The first launch creates `notice.db` (SQLite) and seeds a default admin user.

8.3 First login

From a browser, open:

```
https://<notice-server-ip>:8080
```

Accept the self-signed cert warning. Log in with: - **Username:** `admin` - **Password:** `admin`

IMMEDIATELY change the admin password via **Admin** → **Users** → **admin** → **Change Password**.

8.4 Create additional users

Admin → **Users** → **+ Create User**. Roles: - **admin** — full access (user mgmt, rules, policies, everything) - **analyst** — can work incidents (assign, close, upload evidence, etc.) — cannot manage users/rules/policies - **viewer** — read-only

Assign one admin per SOC lead, analysts per shift member, viewer for auditors / management.

8.5 Register your assets

Admin → **Assets** → **+ Add Asset**. For each machine you want to see labeled in alerts: - IP address - Owner (person or team responsible) - Asset type (workstation / server / IoT / network) - Business criticality (flag critical assets — they get promoted to incidents faster) - Department / notes

Any alert involving a registered asset shows the owner label instead of a bare IP.

8.6 Verify alerts arrive

Wait 2–3 minutes (the pipeline tails `eve.json` every 2s and auto-promote runs periodically). Go to **Alerts** tab — you should see alerts appearing.

If not, check §12 Troubleshooting.

9. Running NOTICE as a Systemd Service

For production, run NOTICE under systemd so it survives reboots and gets restarted on crash.

Create `/etc/systemd/system/notice.service` as root:

```
[Unit]
Description=NOTICE Security Monitor
After=network.target suricata.service
Wants=suricata.service

[Service]
Type=simple
User=notice
Group=suricata
WorkingDirectory=/home/notice/notice
Environment=PATH=/home/notice/notice/venv/bin
EnvironmentFile=/home/notice/notice/.env
ExecStart=/home/notice/notice/venv/bin/python3 /home/notice/notice/app.py
Restart=on-failure
RestartSec=5
StandardOutput=append:/var/log/notice/notice.log
StandardError=append:/var/log/notice/notice.log

# Security hardening
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=false
ReadWritePaths=/home/notice/notice /var/log/notice

[Install]
WantedBy=multi-user.target
```

Prep the log directory:

```
sudo mkdir -p /var/log/notice
sudo chown notice:notice /var/log/notice
```

Enable and start:

```
sudo systemctl daemon-reload
sudo systemctl enable --now notice.service
sudo systemctl status notice.service
```

View logs:

```
sudo tail -f /var/log/notice/notice.log
# or
sudo journalctl -u notice.service -f
```

10. Post-Install Tasks

10.1 Add custom detection rules

Edit `/var/lib/suricata/rules/local.rules` (create if missing) with rules specific to your environment. Example patterns for a corporate LAN:

```
# Detect internal SSH brute force (3+ SYN from same source in 2 min)
alert tcp $HOME_NET any -> $HOME_NET 22 (msg:"NOTICE Internal SSH Brute Force"; flow:to_server; flags:S,12; threshold:type both, track by_src,
count 3, seconds 120; classtype:attempted-admin; sid:9000010; rev:1;)

# Alert if any device connects to a known Tor exit node
# (needs an IP list; use suricata-update for public Tor node lists)

# Alert on plaintext credentials in HTTP POST
alert http $HOME_NET any -> !$HOME_NET any (msg:"NOTICE Plaintext Credentials Sent"; flow:established,to_server; http.method; content:"POST";
http.request_body; content:"password="; nocase; classtype:policy-violation; sid:9000020; rev:1;)
```

After adding rules, validate + reload Suricata:

```
sudo suricata -T -c /etc/suricata/suricata.yaml
sudo systemctl restart suricata
```

10.2 Configure auto-promotion (Alerts → Incidents)

By default, alerts are just alerts. To auto-create incidents when interesting alerts fire, go to **Incidents** → **Smart Rules** → **+ Add Rule**:

Example rules: - **Critical severity alerts hitting critical assets** → auto-promote - **Same signature fired ≥5 times from same source in 10 min** (alert burst) - **Any alert where source IP has a threat-intel score ≥80** (needs VT/AbuseIPDB configured) - **Any signature mapped to MITRE Exploitation phase or later** (kill-chain phase ≥ 4)

Rules are evaluated in priority order; lowest priority number first.

10.3 Register asset baselines

Suricata alone doesn't know what's normal. **Admin** → **Assets** → **each asset** → **Baseline** lets you record: typical outbound flow rate, business hours, expected ports/services. Anomalies against baseline generate alerts.

10.4 Notifications

Admin → **Notifications** — configure webhook or email recipients that receive alerts for specific severity / signature patterns.

Assignment notifications inside NOTICE (bell icon in top nav) are automatic — no config needed.

11. Operational Procedures

11.1 Start / stop / restart

```
sudo systemctl start notice      # start
sudo systemctl stop notice       # stop
sudo systemctl restart notice    # restart
sudo systemctl status notice     # health check

sudo systemctl restart suricata  # reload rules + restart engine
```

11.2 Update Suricata rules (weekly)

```
sudo suricata-update
sudo suricata -c reload-rules  # hot reload if socket enabled
# or
sudo systemctl restart suricata  # cold reload
```

Add to cron for automatic weekly updates:

```
sudo crontab -e
```

Add:

```
0 3 * * 0 /usr/bin/suricata-update && /bin/systemctl restart suricata
```

11.3 Update NOTICE

```
sudo -iu notice
cd ~/notice
git pull origin main
source venv/bin/activate
# If new deps: pip install -r requirements.txt (repo currently uses stdlib only)
sudo systemctl restart notice
```

11.4 Database backup

NOTICE's DB is a single SQLite file at `~/notice/notice.db`.

Manual backup:

```
sudo -iu notice
cd ~/notice
sqlite3 notice.db ".backup notice.db.backup.${date +%Y%m%d}"
```

Automate via cron (nightly at 2am):

```
crontab -e
```

Add:

```
0 2 * * * cd /home/notice/notice && sqlite3 notice.db ".backup notice.db.backup.${date +%Y%m%d}"
0 3 * * * find /home/notice/notice/notice.db.backup.* -mtime +30 -delete
```

11.5 Log rotation

Suricata's `eve.json` grows quickly. Rotate with logrotate:

Create `/etc/logrotate.d/suricata`:

```
/var/log/suricata/eve.json {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 640 suricata suricata
    postrotate
        /bin/kill -HUP $(cat /run/suricata.pid 2>/dev/null) 2>/dev/null || true
    endscript
}
```

NOTICE's pipeline automatically detects log rotation (inode change or truncation) and continues from the start of the new file — no manual work needed.

11.6 Common CLI diagnostics

```
# How many alerts today?
sudo grep -c '"event_type":"alert"' /var/log/suricata/eve.json

# Suricata stats
sudo cat /var/log/suricata/stats.log | tail -30

# NOTICE pipeline lag (how far behind eve.json is the DB)
cd ~/notice && source venv/bin/activate && python3 -c "
from db import get_db
c = get_db()
r = c.execute("SELECT byte_offset, updated_at FROM pipeline_state WHERE source='eve.json'").fetchone()
print(f'Bookmark: {r["byte_offset"]:,} bytes, updated {r["updated_at"]}')
import os
size = os.path.getsize('/var/log/suricata/eve.json')
print(f'eve.json size: {size:,} bytes ({size - r["byte_offset"]:,} bytes behind)')
"
```

12. Troubleshooting

Suricata is running but no alerts appear

Check 1 — Is Suricata seeing packets?

```
sudo suricata -c iface-stat <MON_IFACE>
# Look for: "pkts": <a large growing number>
```

If `pkts` is 0 or growing slowly, the mirror port isn't feeding you traffic. Re-verify with `tcpdump -i <MON_IFACE>`.

Check 2 — Are the rules loaded?

```
sudo grep -c '^alert' /var/lib/suricata/rules/suricata.rules
```

Should be $\geq 20,000$ with ET Open ruleset enabled.

Check 3 — Any packet loss?

```
sudo grep 'invalid_checksum\|packet.drops' /var/log/suricata/stats.log | tail
```

If `invalid_checksum` grows fast, ensure `checksum-validation: no` in `suricata.yaml` (step 5.3.3).

Check 4 — Generate a test alert:

```
# from any host that reaches the internet through your monitored network
curl http://testmyids.com/
```

Within 30 seconds, this should trigger `GPL ATTACK_RESPONSE id check returned root` or similar in `eve.json`.

NOTICE UI loads but shows “No alerts”

Check pipeline state:

```
sudo tail -20 /var/log/notice/notice.log
```

If pipeline is stuck, restart the service:

```
sudo systemctl restart notice
```

Check the pipeline bookmark isn’t past the end of `eve.json` (happens after log rotation without inode change):

```
cd ~/notice && source venv/bin/activate && python3 -c "
from db import get_db
import os
c = get_db()
r = c.execute("SELECT byte_offset FROM pipeline_state WHERE source='eve.json'").fetchone()
size = os.path.getsize('/var/log/suricata/eve.json')
print(f'bookmark: {r["byte_offset"]}, / size: {size},}')
if r['byte_offset'] > size:
    c.execute("UPDATE pipeline_state SET byte_offset=0 WHERE source='eve.json'")
    c.commit()
    print('Bookmark reset – pipeline will re-ingest.')
"
```

“Too many login attempts” lockout

NOTICE rate-limits failed login attempts (5 fails → 15 min lockout). Unlock by restarting the service (in-memory tracker resets):

```
sudo systemctl restart notice
```

For persistent unlock or to reset the admin password:

```
sudo -iu notice
cd ~/notice && source venv/bin/activate
python3 -c "
from db import get_db
from auth import hash_password
h, s = hash_password('YourNewStrongPassword')
c = get_db()
c.execute("UPDATE users SET password_hash=?, salt=?, active=1 WHERE username=\"admin\"", (h, s))
c.commit()
print('admin password reset')
"
```

Browser back button logs me out

The current code uses `history.pushState` for tab navigation, so browser Back returns to the previous tab (not out of NOTICE). If you’re on the very first NOTICE page and press Back, it goes to whatever URL preceded NOTICE (which may look like a logout). This is expected browser behavior.

Can’t login from another machine on the network

Check that: 1. The NOTICE server’s firewall allows port 8080 from that subnet: `bash sudo ufw allow from 10.0.0.0/8 to any port 8080 proto tcp` 2. You’re using HTTPS: `https://<server-ip>:8080` (not `http://`) 3. The browser accepts the self-signed cert (or use a proper CA cert)

Suricata rule loading fails with syntax errors

```
sudo suricata -T -c /etc/suricata/suricata.yaml 2>&1 | grep -i error
```

Fix each rule referenced in the errors, or comment out with `#`. Restart Suricata.

13. Security Hardening

Before exposing NOTICE beyond a trusted network:

13.1 Change all default credentials

- Admin password (done in §8.3 already).
- Add unique passwords for every analyst / viewer account.

13.2 Use a proper TLS certificate

Self-signed certs are fine internally but browsers will warn. Options:

Option A — Let's Encrypt via reverse proxy (recommended)

Install nginx as a reverse proxy in front of NOTICE:

```
sudo apt install -y nginx certbot python3-certbot-nginx
```

Create `/etc/nginx/sites-available/notice`:

```
server {
    listen 443 ssl http2;
    server_name notice.example.com;

    ssl_certificate      /etc/letsencrypt/live/notice.example.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/notice.example.com/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;

    location / {
        proxy_pass https://127.0.0.1:8080;
        proxy_ssl_verify off;                # self-signed on backend
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_read_timeout 300s;
    }
}
```

Enable, obtain cert, restart:

```
sudo ln -s /etc/nginx/sites-available/notice /etc/nginx/sites-enabled/
sudo certbot --nginx -d notice.example.com
sudo systemctl reload nginx
```

Now users connect to `https://notice.example.com` and get a valid cert.

13.3 Firewall

Restrict port 8080 (or your reverse-proxy port) to trusted networks only:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp comment 'SSH'
sudo ufw allow from 10.0.0.0/8 to any port 8080 proto tcp comment 'NOTICE UI'
sudo ufw enable
```

Never expose NOTICE directly to the public internet without additional authentication (VPN, mTLS, reverse-proxy auth).

13.4 Enforce role separation

Default RBAC: - **viewer** — read-only, cannot POST/PUT/DELETE anything - **analyst** — full incident/evidence/IOC ops; blocked from user mgmt, rule mgmt, policy mgmt - **admin** — everything

Assign roles thoughtfully. Never give analysts admin unless they also administer the platform.

13.5 Regular backups

Follow §11.4 for daily DB backups. Store backups off-server.

13.6 Rotate API keys periodically

If you use VirusTotal or AbuseIPDB, rotate the keys quarterly and update `.env`. NOTICE reads them from env on start; restart the service after updating.

13.7 Monitor NOTICE itself

Add NOTICE and Suricata log paths to your existing SIEM / log aggregator if you have one. NOTICE is a security tool — if it crashes silently, you're blind.

14. Optional: External Enrichment

14.1 VirusTotal (free tier: 500 lookups/day)

1. Sign up at <https://www.virustotal.com> — get your API key.
2. Add to `.env`:

```
VIRUSTOTAL_API_KEY=your-key-here
```

3. Restart NOTICE: `sudo systemctl restart notice`.

External IPs in alerts will now be enriched with VT reputation scores. Cached to avoid quota exhaustion.

14.2 AbuseIPDB (free tier: 1000 checks/day)

1. Sign up at <https://www.abuseipdb.com> — get your API key.
2. Add to `.env`:

```
ABUSEIPDB_KEY=your-key-here
```

3. Restart NOTICE.

External IPs get an abuse confidence score (0–100). Cached similarly.

14.3 GeoIP (offline database — free)

For country/city/ISP enrichment of external IPs without any API calls:

```
sudo -iu notice
cd ~/notice
mkdir -p geolite2
# Get a MaxMind free account and download GeoLite2-City.mmdb
# Place it in: geolite2/GeoLite2-City.mmdb
```

Restart NOTICE. External IPs will now show country/city/ISP badges.

Appendix: File Layout Reference

```
~/notice/
├── app.py                # main entry point (Bottle server + daemon threads)
├── db.py                 # schema, seed data, connection helpers
├── auth.py               # sessions, RBAC decorators, password hashing
├── eve_reader.py         # eve.json tail + noise filter
├── pipeline.py           # DB ingestion pipeline (daemon thread)
├── analyzers/            # correlation, TI, auto-promote, anomaly, GeoIP
├── routes/               # HTTP endpoints (~40 modules)
├── static/index.html     # single-page app frontend
├── certs/                # TLS cert + key (gitignored)
├── evidence/             # uploaded POC files per incident (gitignored)
├── geolite2/             # offline GeoIP DB (gitignored)
├── notice.db             # SQLite database (gitignored)
├── venv/                 # Python virtualenv (gitignored)
└── .env                  # config (gitignored)
```

Suricata layout:

```
/etc/suricata/suricata.yaml      # main config
/var/lib/suricata/rules/
├── suricata.rules              # merged rules from suricata-update
└── local.rules                 # your custom rules
/var/log/suricata/
├── eve.json                   # JSON events (NOTICE reads this)
├── suricata.log                # engine log
├── stats.log                  # perf counters
└── log.pcap                   # rolling packet capture (optional)
```

Support

- **Repo:** <https://github.com/credninja/notice>
- **Suricata docs:** <https://docs.suricata.io>
- **File issues:** open on GitHub

Good luck. If you follow this guide end-to-end, you should have a working SOC console in under 2 hours.